

What Are Variables and Data Types in Java?

From Zero to Real Programs - Simple Explanation, Deep Concepts, Hands-On Labs, Exam Answers and Interview Preparation

Layman definition: A variable is a named container that stores a value. A data type tells Java what kind of value the container may hold and what operations are valid.



What you will learn

- How to **declare, initialize, assign and read** variables
- The difference between **primitive and reference data types**
- All 8 Java primitive types with size, range, examples and default field values
- Integer overflow, floating-point precision, Unicode characters and booleans
- String, arrays, classes, wrapper types and null references
- Local, instance and static variables with real-world examples
- Widening and narrowing type casting
- The correct use of **var** and **final**
- Hands-on labs using Scanner, odd/even logic, price calculation and overflow experiments
- Exam, viva and interview answers, common errors and practice tasks

Memory formula: Variable = name + data type + stored value. Example: `int age = 20;`

1. The Core Idea: What Is a Variable?

A variable is a named storage location used by a program. It allows the value to be read, changed and reused while the program runs.

Real-life analogy: labeled containers

Container label	What it stores	Java example
age	A whole number	int age = 20;
price	A decimal value	double price = 99.50;
grade	One character	char grade = 'A';
isLoggedIn	True or false	boolean isLoggedIn = true;
studentName	Text	String studentName = "Riya";

Important: The variable name is the label. The value is the content. The data type is the rule describing what may be stored.

Why do programs need variables?

- To remember user input such as name, age and marks
- To perform calculations such as total, average, tax and discount
- To track state such as logged in, paid, delivered or active
- To pass values between methods
- To model real objects such as students, bank accounts and orders

A variable may change

Example

```
int score = 40;
score = 55;
score = score + 10;

System.out.println(score); // 65
```

The name remains score, but the stored value changes from 40 to 55 and then to 65.

2. Variable Syntax: Declaration, Initialization and Assignment

```
dataType variableName = value;

int age = 20;
double price = 149.99;
boolean active = true;
```

Part	Example	Meaning
Data type	int	The allowed kind of value
Variable name	age	The name used to access the value
Assignment operator	=	Stores the right-side value in the variable
Value	20	The actual data
Statement terminator	;	Ends the Java statement

Four actions you must know

Action	Code	Meaning
Declaration	int marks;	Create the variable name and type
Initialization	marks = 75;	Give the variable its first value
Declaration + initialization	int marks = 75;	Do both in one statement
Reassignment	marks = 90;	Replace the old value
Reading	System.out.println(marks);	Use the current value

Correct naming rules

- A name may use letters, digits, underscore and dollar sign.
- A name cannot start with a digit.
- Java keywords such as int, class and public cannot be variable names.
- Names are case-sensitive: age and Age are different.
- Use meaningful camelCase names: totalPrice, studentName, isActive.

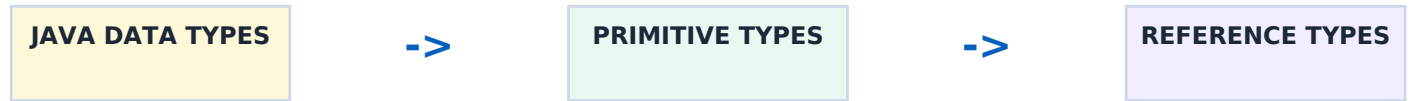
```
// Good
int studentAge = 20;
double totalPrice = 499.75;
boolean isPaymentComplete = false;

// Poor
int a = 20;
double x = 499.75;
```

Professional habit: A meaningful variable name reduces the number of comments needed and makes debugging easier.

3. Java Data Types: The Big Picture

A data type tells Java how a value should be interpreted, what operations are valid and how the value is represented.



Category	Examples	What the variable contains
Primitive	byte, short, int, long, float, double, char, boolean	The primitive value itself
Reference	String, arrays, classes, interfaces, enums	A reference to an object or null

The 8 primitive data types

- **Whole numbers:** byte, short, int, long
- **Decimal numbers:** float, double
- **Single character:** char
- **Logical value:** boolean

Reference type examples

```
String name = "Pranames";
int[] marks = {80, 75, 92};
Student student = new Student();
String message = null;
```

Do not confuse: String is not a primitive type. It is a class and therefore a reference type.

Compile-time type safety

```
int age = 20;

// Compile-time error:
// age = "twenty";
```

Because age is declared as int, Java prevents text from being assigned to it. This catches many mistakes before the program runs.

4. Primitive Types: Complete Reference Table

Type	Size	Typical range / values	Example literal	Default field value
byte	8-bit	-128 to 127	100	0
short	16-bit	-32,768 to 32,767	30000	0
int	32-bit	-2,147,483,648 to 2,147,483,647	100000	0
long	64-bit	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	9_000_000_000L	0L
float	32-bit	Approx. 6-7 decimal digits precision	3.14F	0.0F
double	64-bit	Approx. 15-16 decimal digits precision	3.14159	0.0D
char	16-bit	Unsigned UTF-16 code unit: 0 to 65,535	'A'	'\u0000'
boolean	Implementation-dependent storage	true or false	true	false

Important: The default values shown above apply to instance fields and static fields. Local variables do not receive automatic default values.

Which numeric type should a beginner choose?

Need	Recommended starting type	Reason
Normal whole-number calculation	int	Most common integer type
Very large whole number	long	Larger range than int
Normal decimal calculation	double	Default floating-point type
Memory-sensitive binary/file data	byte	Small 8-bit range
Money requiring exact decimal rules	BigDecimal	Avoids binary floating-point rounding problems

Interview caution: Do not say boolean is exactly 1 bit in Java. The language defines true and false, but storage representation is implementation-dependent.

5. Integer Types: byte, short, int and long

Integer types store whole numbers without a fractional part. Java uses signed two's-complement representation for byte, short, int and long.

Common examples

```
byte temperature = 35;
short year = 2026;
int population = 1_500_000;
long worldPopulation = 8_100_000_000L;
```

Why does long often need L?

```
long valid = 3_000_000_000L;

// Without L, this literal is treated as int first
// and is too large for int:
// long invalid = 3_000_000_000;
```

Integer overflow experiment

```
int maximum = Integer.MAX_VALUE;
System.out.println(maximum);      // 2147483647

int overflowed = maximum + 1;
System.out.println(overflowed);   // -2147483648
```

Surprising result: Integer overflow normally wraps around. Java does not automatically throw an exception for ordinary int addition overflow.

Detecting overflow when needed

```
int result = Math.addExact(Integer.MAX_VALUE, 1);
// Throws ArithmeticException instead of silently wrapping.
```

byte and short arithmetic

```
byte a = 10;
byte b = 20;

// Arithmetic is promoted to int:
// byte sum = a + b; // compile-time error

int sum = a + b;      // correct
byte smallSum = (byte) (a + b); // explicit narrowing
```

6. Decimal Types: float and double

float and double store IEEE 754 binary floating-point values. They are fast and useful for scientific, measurement and graphics calculations, but many decimal fractions cannot be represented exactly in binary.

Examples

```
float height = 5.8F;  
double pi = 3.141592653589793;  
double temperature = 36.6;
```

Why float needs F

```
// Decimal literals are double by default:  
float rate = 2.5F;  
  
// Compile-time error without F:  
// float wrong = 2.5;
```

Precision experiment

```
double result = 0.1 + 0.2;  
System.out.println(result);  
// Often prints: 0.30000000000000004
```

This is not a Java bug: It is a consequence of representing decimal fractions using binary floating-point.

For money: prefer BigDecimal

```
import java.math.BigDecimal;  
  
BigDecimal price = new BigDecimal("0.10");  
BigDecimal tax = new BigDecimal("0.20");  
BigDecimal total = price.add(tax);  
  
System.out.println(total); // 0.30
```

Professional rule: For exact currency calculations, use BigDecimal with string input and an explicit rounding policy.

7. char and boolean

char: one UTF-16 code unit

```
char grade = 'A';  
char digit = '7';  
char symbol = '@';  
char unicodeA = '\u0041';  
  
System.out.println(unicodeA); // A
```

- A char literal uses **single quotes**: 'A'.
- A String literal uses **double quotes**: "A".
- char is numeric internally and can participate in arithmetic.
- Some Unicode characters require two char values called a surrogate pair.

```
char letter = 'A';  
int code = letter;  
  
System.out.println(code); // 65
```

Unicode caution: char does not always represent one complete visible Unicode character. Some characters, including many emoji, use two UTF-16 code units.

boolean: logical state

```
boolean isLoggedIn = true;  
boolean hasPermission = false;  
  
if (isLoggedIn && hasPermission) {  
    System.out.println("Access granted");  
} else {  
    System.out.println("Access denied");  
}
```

- boolean accepts only true or false.
- Java does not treat 0 as false or 1 as true.
- Use positive names such as isActive, hasAccess and paymentComplete.

8. Reference Types: String, Arrays and Objects

A reference variable identifies an object or contains null. Reference types include classes, arrays, interfaces, enums and records.

String example

```
String name = "Anita";  
String message = new String("Hello");  
  
System.out.println(name.length());  
System.out.println(name.toUpperCase());
```

Array example

```
int[] marks = {80, 92, 75};  
  
System.out.println(marks[0]); // 80  
System.out.println(marks.length); // 3
```

Custom object example

```
class Student {  
    String name;  
    int marks;  
}  
  
Student first = new Student();  
first.name = "Ravi";  
first.marks = 88;
```

null means no object reference

```
String text = null;  
  
// Runtime error: NullPointerException  
// System.out.println(text.length());
```

Primitive vs reference: A primitive variable directly holds a primitive value. A reference variable refers to an object or contains null.

Reference assignment shares the same object

```
int[] first = {10, 20};  
int[] second = first;  
  
second[0] = 99;  
System.out.println(first[0]); // 99
```

Both variables refer to the same array object. Changing the object through one reference is visible through the other.

9. Primitive Types vs Wrapper Classes

Primitive	Wrapper class	Example use
byte	Byte	Collections and utility methods
short	Short	Object form of short
int	Integer	List<Integer>, parsing numbers
long	Long	Large-number object
float	Float	Object form of float
double	Double	Object form of double
char	Character	Character utility methods
boolean	Boolean	Nullable logical object

Why wrappers exist

- Java collections store objects, not primitive type parameters.
- Wrappers provide methods such as Integer.parseInt.
- Wrapper references can be null.
- Generics use types such as List<Integer>, not List<int>.

Autoboxing and unboxing

```
Integer objectNumber = 10; // autoboxing: int -> Integer
int primitiveNumber = objectNumber; // unboxing: Integer -> int
```

Danger: null unboxing

```
Integer value = null;

// Runtime NullPointerException:
// int number = value;
```

Do not compare wrapper objects using ==

```
Integer a = 1000;
Integer b = 1000;

System.out.println(a == b); // may be false
System.out.println(a.equals(b)); // true
```

Interview answer: Use == for primitive value comparison. For wrapper object content, use equals unless identity comparison is intentionally required.

10. Variable Categories and Scope

Variable category	Declared where?	Lifetime / ownership	Example
Local variable	Inside method, constructor or block	Exists during that execution	int total = 0;
Parameter	Method or constructor parameter list	Local to that call	void print(int value)
Instance field	Inside class, outside methods, without static	One copy per object	String name;
Static field	Inside class with static	One copy per class	static int count;

Real-world class example

```
class BankAccount {
    static String bankName = "Safe Bank"; // static field

    String accountHolder; // instance field
    double balance;        // instance field

    void deposit(double amount) { // parameter
        double oldBalance = balance; // local variable
        balance = balance + amount;
        System.out.println(oldBalance + " -> " + balance);
    }
}
```

Block scope

```
if (true) {
    int inside = 10;
    System.out.println(inside);
}

// Compile-time error: inside is out of scope here
// System.out.println(inside);
```

Remember: Scope answers: Where can the variable name be used? Lifetime answers: How long does its storage/value exist?

11. Default Values and Definite Assignment

Fields receive automatic default values. Local variables must be definitely assigned before Java allows them to be read.

```
class Example {
    int count;           // default 0
    double price;        // default 0.0
    boolean active;      // default false
    String name;         // default null
}
```

Local variable has no automatic default

```
public class Main {
    public static void main(String[] args) {
        int marks;

        // Compile-time error:
        // System.out.println(marks);

        marks = 80;
        System.out.println(marks); // correct
    }
}
```

Type of variable	Automatic default?	Must assign before reading?
Instance field	Yes	No
Static field	Yes	No
Local variable	No	Yes
Parameter	Provided by caller	Already has a value

Why Java does this: Requiring local variables to be assigned before use prevents accidental calculations with unknown values.

12. Literals and Readable Numeric Values

Integer literal forms

```
int decimal = 255;
int binary = 0b1111_1111;
int octal = 0377;
int hexadecimal = 0xFF;

System.out.println(decimal == binary); // true
```

Underscores improve readability

```
int oneMillion = 1_000_000;
long cardNumber = 1234_5678_9012_3456L;
double amount = 10_000.50;
```

- Do not place an underscore at the beginning or end of a literal.
- Do not place it directly next to a decimal point or suffix.
- Underscores change readability, not value.

Special floating-point values

```
double positiveInfinity = 1.0 / 0.0;
double notANumber = 0.0 / 0.0;

System.out.println(positiveInfinity); // Infinity
System.out.println(notANumber);       // NaN
System.out.println(Double.isNaN(notANumber)); // true
```

Character escape sequences

Literal	Meaning
'\n'	New line
'\t'	Tab
'\\'	Backslash
'\"'	Single quote
"Hello\nJava"	String containing a line break

13. Type Conversion and Casting

Widening conversion: smaller to larger compatible type

```
int quantity = 50;
long largeQuantity = quantity;
double decimalQuantity = largeQuantity;

System.out.println(decimalQuantity); // 50.0
```

Widening numeric conversion is usually automatic because the destination type can represent at least the source type's range, although converting large integers to floating-point can still lose precision.

Narrowing conversion: larger to smaller type

```
double price = 99.99;
int wholePrice = (int) price;

System.out.println(wholePrice); // 99
```

Narrowing risk: A cast can discard fractional data or change a value when it is outside the destination type's range.

Overflow during narrowing

```
int value = 130;
byte small = (byte) value;

System.out.println(small); // -126
```

char and int conversion

```
char letter = 'A';
int codePointUnit = letter; // 65

char next = (char) (letter + 1);
System.out.println(next); // B
```

String conversion

```
String input = "123";

int number = Integer.parseInt(input);
String text = String.valueOf(number);

System.out.println(number + 1); // 124
```

14. var and final

Local variable type inference with var

```
var age = 20;           // inferred as int
var name = "Riya";      // inferred as String
var marks = new int[]{80, 90}; // inferred as int[]
```

- var may be used for local variables with an initializer.
- var is not a dynamic type. The inferred type is fixed at compile time.
- var cannot be used without enough information, such as `var value = null`.
- var cannot be used for fields, method parameters or return types.

```
var number = 10;

// Compile-time error because number is int:
// number = "ten";
```

Use var carefully: Use it when the inferred type is obvious. Avoid it when it hides important information from the reader.

Constants with final

```
final double GST_RATE = 0.18;
final int MAX_LOGIN_ATTEMPTS = 3;

// Compile-time error:
// GST_RATE = 0.20;
```

- A final variable may be assigned only once.
- Constants are commonly named in UPPER_SNAKE_CASE.
- final prevents reassignment of a reference, not necessarily mutation of the referenced object.

```
final int[] values = {10, 20};

values[0] = 99; // allowed: object contents changed
// values = new int[]{1, 2}; // not allowed: reference reassignment
```

15. Hands-On Lab 1: Student Profile

Goal

Store different kinds of student information, calculate an average and print a formatted result.

```
public class StudentProfile {  
    public static void main(String[] args) {  
        String name = "Ananya";  
        int age = 19;  
        char grade = 'A';  
        boolean scholarship = true;  
  
        double mathematics = 84.5;  
        double programming = 92.0;  
        double average = (mathematics + programming) / 2;  
  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
        System.out.println("Grade: " + grade);  
        System.out.println("Scholarship: " + scholarship);  
        System.out.println("Average: " + average);  
    }  
}
```

What this lab demonstrates

- String reference variable
- int, char, boolean and double primitive variables
- Arithmetic expression and assignment
- Reading values using concatenation

Experiment

1. Change the student's values and run again.
2. Replace double with int for marks and observe the average.
3. Print whether the average is greater than or equal to 90.
4. Add a final constant named COLLEGE_NAME.

16. Hands-On Lab 2: User Input and Odd/Even

```
import java.util.Scanner;

public class OddEvenInput {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter a whole number: ");
        int number = scanner.nextInt();

        boolean isEven = number % 2 == 0;

        System.out.println("Number: " + number);
        System.out.println("Is even: " + isEven);

        if (isEven) {
            System.out.println("Result: EVEN");
        } else {
            System.out.println("Result: ODD");
        }

        scanner.close();
    }
}
```

Compile and run

```
javac OddEvenInput.java
java OddEvenInput
```

Variable map

Variable	Type	Purpose
scanner	Scanner	Reads keyboard input
number	int	Stores the user's whole number
isEven	boolean	Stores the condition result

Test cases

Input	number % 2	isEven	Output
8	0	true	EVEN
7	1	false	ODD
-4	0	true	EVEN
0	0	true	EVEN

17. Hands-On Lab 3: Shopping Bill and Type Casting

```
public class ShoppingBill {  
    public static void main(String[] args) {  
        int quantity = 3;  
        double unitPrice = 149.75;  
        final double TAX_RATE = 0.05;  
  
        double subtotal = quantity * unitPrice;  
        double tax = subtotal * TAX_RATE;  
        double total = subtotal + tax;  
  
        int roundedDownTotal = (int) total;  
  
        System.out.println("Subtotal: " + subtotal);  
        System.out.println("Tax: " + tax);  
        System.out.println("Total: " + total);  
        System.out.println("Whole-number total: " + roundedDownTotal);  
    }  
}
```

Concepts used

- int quantity and double unitPrice
- final constant TAX_RATE
- Automatic numeric promotion in quantity * unitPrice
- Explicit narrowing cast from double to int
- Loss of fractional data after narrowing

Real billing warning: This example teaches types. Production financial systems should define exact rounding rules and commonly use BigDecimal.

Challenge

1. Add a discount percentage.
2. Reject a negative quantity.
3. Use BigDecimal for an exact-money version.
4. Format the total to two decimal places.

18. Hands-On Lab 4: Overflow and Precision Experiments

Experiment A: int overflow

```
public class OverflowLab {
    public static void main(String[] args) {
        int value = Integer.MAX_VALUE;

        System.out.println(value);
        System.out.println(value + 1);

        try {
            System.out.println(Math.addExact(value, 1));
        } catch (ArithmeticException exception) {
            System.out.println("Overflow detected");
        }
    }
}
```

Experiment B: decimal precision

```
public class PrecisionLab {
    public static void main(String[] args) {
        double a = 0.1;
        double b = 0.2;

        System.out.println(a + b);
        System.out.println((a + b) == 0.3);
    }
}
```

Questions to answer

1. Why does int wrap after Integer.MAX_VALUE?
2. Why can $(0.1 + 0.2) == 0.3$ be false?
3. Which problem is solved by Math.addExact?
4. Which type should be considered for exact decimal money?

Learning result: Choosing a type is not only about syntax. It also determines range, precision, allowed operations and possible failure modes.

19. Common Errors and How to Fix Them

Problem	Wrong idea / code	Fix
Uninitialized local variable	<code>int x; print(x);</code>	Assign x before reading it
Wrong literal suffix	<code>float f = 2.5;</code>	Use 2.5F
Large integer literal	<code>long n = 30000000000;</code>	Use 30000000000L
Text in primitive variable	<code>int age = "20";</code>	Parse text or use int literal
Lossy conversion	<code>int n = 9.8;</code>	Use double or explicit cast
char vs String	<code>char c = "A";</code>	Use 'A' for char
Wrapper null unboxing	<code>Integer x = null; int y = x;</code>	Check null before unboxing
String comparison	<code>name == "Riya"</code>	Use <code>name.equals("Riya")</code>
Overflow	<code>MAX_VALUE + 1</code>	Validate range or use exact methods / larger type
Money with double	<code>0.1 + 0.2</code>	Use BigDecimal for exact decimal rules

Compiler messages are useful

Do not hide compiler errors. Read the first useful message, check the line number and understand whether the problem is type mismatch, missing initialization, invalid literal or scope.

A simple debugging checklist

1. What is the declared type?
2. What is the actual value or expression type?
3. Is the variable in scope?
4. Has it been assigned before reading?
5. Can range or precision be lost?
6. Is this primitive comparison or object comparison?

20. Exam, Viva and Interview Answers

2-mark answer: What is a variable?

A variable is a named storage location used to hold a value during program execution. Every Java variable has a declared or inferred type that controls the values and operations allowed.

2-mark answer: What is a data type?

A data type classifies a value and tells Java how it may be stored, interpreted and used. Java data types are broadly primitive types and reference types.

5-mark answer

A Java variable is a named location used to store data. A variable is declared with a type and a name, and it may be initialized with a value. Java has eight primitive types: byte, short, int, long, float, double, char and boolean. Reference types include String, arrays, classes, interfaces and other object types. The type system provides compile-time checking, controls valid operations and affects range, precision and memory representation. Variables may be local variables, parameters, instance fields or static fields. Local variables must be assigned before use, while fields receive default values.

Fast viva answers

Question	Answer
How many primitive types are in Java?	Eight
Default integer type?	int
Default decimal literal type?	double
Is String primitive?	No, it is a class/reference type
Can boolean store 0 or 1?	No, only true or false
Do local variables get defaults?	No
What is widening?	Conversion to a wider compatible numeric type
What is narrowing?	Conversion to a smaller type using an explicit cast
What does final do?	Prevents variable reassignment after initialization
Is var dynamic typing?	No, its type is inferred and fixed at compile time

Popular interview questions

- Primitive type vs reference type
- int vs Integer
- float vs double
- Why is String not primitive?
- Local, instance and static variables
- Default values in Java
- Widening vs narrowing casting
- Autoboxing and unboxing
- Integer overflow
- Why BigDecimal is preferred for exact money
- What var does and does not do
- final reference vs immutable object

21. Practice Tasks and Quick Revision

Practice tasks

1. Create variables for a product: id, name, price, stock quantity and active status.
2. Read two int values with Scanner and print their sum, difference and product.
3. Read Celsius temperature and convert it to Fahrenheit using double.
4. Create a program that demonstrates widening from int to double.
5. Create a program that demonstrates narrowing from double to int.
6. Print Integer.MIN_VALUE, Integer.MAX_VALUE, Long.MIN_VALUE and Long.MAX_VALUE.
7. Use a char and print its numeric value.
8. Create an array reference, copy it to another reference and observe shared mutation.
9. Create a final constant for maximum login attempts.
10. Rewrite one lab using var only where the type remains obvious.

One-page revision table

Concept	Remember
Variable	Named container for a value
Data type	Controls allowed values and operations
Primitive	Value directly stored: 8 built-in types
Reference	Refers to object or null
Local variable	Must be assigned before use
Field	Receives default value
int	Normal whole number
double	Normal decimal calculation
char	One UTF-16 code unit
boolean	true or false
Widening	Usually automatic
Narrowing	Explicit cast; may lose data
var	Local type inference, not dynamic typing
final	Cannot be reassigned after initialization

Final memory formula: Choose a meaningful name, choose the correct type, assign a valid value, and understand range, precision, scope and lifetime.

Teach it in one minute

A variable is a named container, and a data type is the rule for that container. Primitive types directly store simple values such as numbers, characters and booleans. Reference variables refer to objects such as Strings, arrays and custom classes. The chosen type affects valid operations, range, precision and program safety.

22. Technical Notes and Further Learning

This guide focuses on stable Java language concepts. For exact formal details, consult the Java Language Specification and official Java API documentation.

Official topic	What to study
Java Language Specification - Types, Values and Variables	Primitive/reference types, variables, values and conversions
Java Language Specification - Lexical Structure	Literals, identifiers and keywords
Java Language Specification - Conversions and Contexts	Widening, narrowing, boxing and unboxing
Java API - wrapper classes	Integer, Double, Character, Boolean and parsing utilities
Java API - BigDecimal	Exact decimal arithmetic and rounding
Java API - Scanner	Reading typed input

Shareable closing line: Do not memorize only the primitive table. Run the labs, change the values, cause errors intentionally and explain why Java accepts or rejects each line.